Using Convolutional Neural Networks to Identify Human-Made or AI-Generated Images

2024 Taiwan International Science Fair
Research Report

Hao-Hsiang Hung
Taipei Municipal Chien Kuo High School
Instructor: Ting-Chung Wang

Category: Computer Science and Information Engineering

Keywords: CNN, AI images, generation identification

Abstract

With the advancement of image generation techniques, the accuracy of platforms that previously relied on visualizing noise to determine whether an image was generated by AI has declined. Therefore, the purpose of this study is to utilize convolutional neural networks to discern AI-generated images by identifying imperceptible noise. The experiment is divided into three main parts: training using L1 and L2 regularization and dynamic learning rates separately, then simultaneously adding regularization and dynamic learning rates to the convolutional neural network to find the optimal parameters, and finally comparing with existing AI recognition platforms.

I. Research Motivation

With the advancement of image-generation technology, more and more image-generation platforms in recent years — such as Stable Diffusion and MidJourney — can produce images that are indistinguishable from real ones to the naked eye. Although there are currently platforms such as Optic AI or Not that use visualized noise to judge whether an image is AI-generated, as generation technology improves, the visualized noise becomes increasingly difficult to detect, and the accuracy of such platforms declines. Therefore, we wanted to use convolutional neural networks to detect non-visualized noise in order to increase detection accuracy.

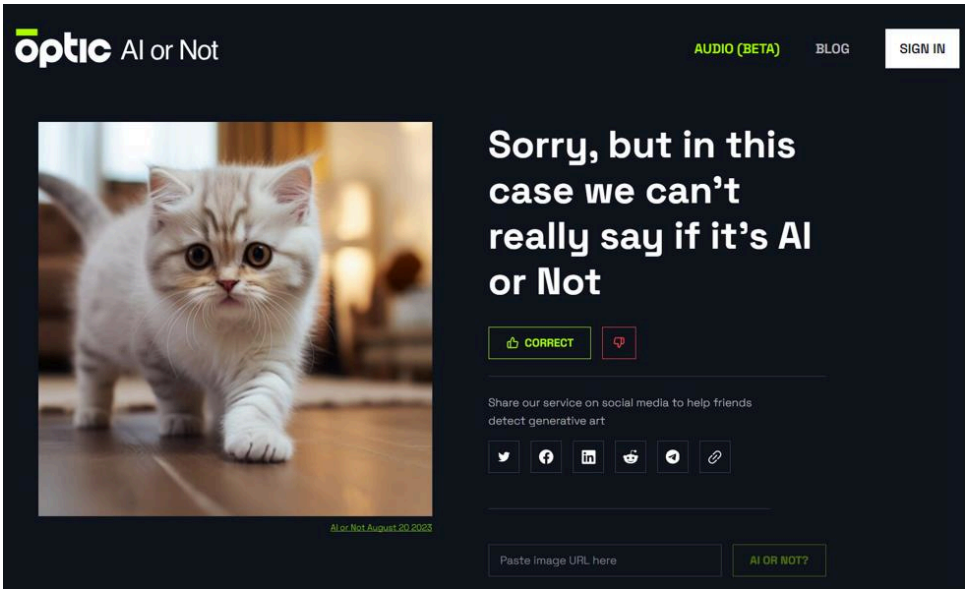


Figure 1. The Optic AI or Not platform for judging AI-generated images

II. Research Purpose

Currently, most platforms for detecting real versus fake images rely on visualized noise to judge whether an image is AI-generated. However, much of the non-visualized noise is imperceptible to the naked eye. Therefore, this experiment uses a convolutional neural network to detect non-visualized noise within images in order to determine whether an image is AI-generated, and compares the results with existing AI image-detection websites. This project has three main goals:

1. Study a convolutional neural network model for distinguishing AI-generated images from human-made images.

2. Vary the regularization parameters and dynamic learning rate to find the optimal model.

3. Compare the results with existing AI image-detection websites.

III. Research Process and Methods

(1) Equipment and Environment

1. Hardware:

RAM: 16GB

CPU: Intel Core i5-4440 3.1GHz

GPU: NVIDIA GeForce RTX 3060

2. Software environment:

Python: Anaconda3

Deep learning framework: TensorFlow

Libraries: Keras, NumPy, cv2, Matplotlib

(2) Research Process

1. Data Collection: The CIFake Dataset (Real and AI-Generated Synthetic Images)

The CIFake dataset contains two labeled classes — synthetic images and real images — and consists of 60,000 synthetic images plus 60,000 real images. The 60,000 real images were collected from CIFAR-10, and the other 60,000 synthetic images were generated using Stable Diffusion version 1.4.

The CIFake dataset was obtained from the Kaggle platform. Although it contains 120,000 images in total, due to insufficient hardware memory, this study used only 5,000 synthetic images and 5,000 real images (10,000 images total) for training.

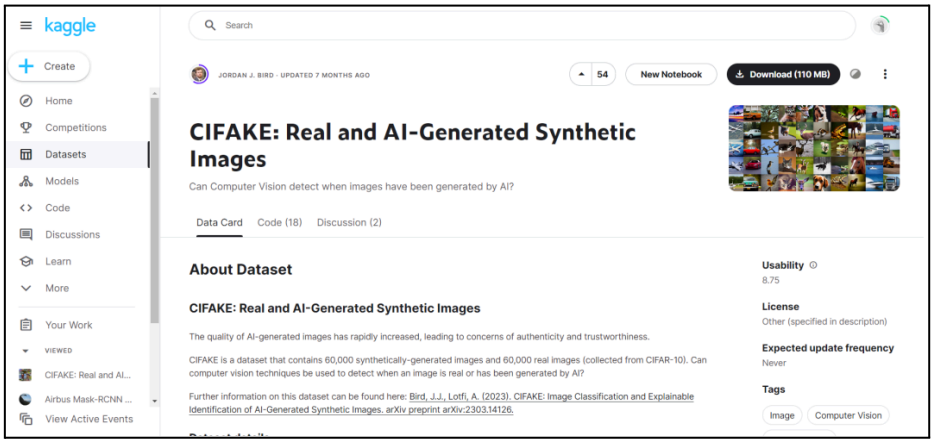


Figure 2. The CIFake dataset on the Kaggle platform [2]

2. Image Preprocessing

To obtain higher resolution and reveal finer feature patterns, the original 32×32 images were resized to 80×80. The images were then normalized by dividing each pixel value by 255 so that pixel values fall between 0 and 1, which speeds up training convergence.

Normalization formula: $x_norm = (x - x_min) / (x_max - x_min)$

(where x is the original feature data, x_norm is the normalized data between 0 and 1, and x_min and x_max are the minimum and maximum values of the feature data, respectively.)

3. Building the Image Recognition Model: VGG16

This study adopted the VGG16 model, adding regularization to the first convolutional layer and to the fully connected layers. The model structure is as follows [1]:

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 80, 80, 64)	1,792
activation (Activation)	(None, 80, 80, 64)	0
conv2d_1 (Conv2D)	(None, 80, 80, 64)	36,928
activation_1 (Activation)	(None, 80, 80, 64)	0
max_pooling2d (MaxPooling2D)	(None, 40, 40, 64)	0
conv2d_2 (Conv2D)	(None, 40, 40, 128)	73,856
activation_2 (Activation)	(None, 40, 40, 128)	0
conv2d_3 (Conv2D)	(None, 40, 40, 128)	147,584
activation_3 (Activation)	(None, 40, 40, 128)	0
max_pooling2d_1 (MaxPooling2D)	(None, 20, 20, 128)	0
conv2d_4 (Conv2D)	(None, 20, 20, 256)	295,168
activation_4 (Activation)	(None, 20, 20, 256)	0
conv2d_5 (Conv2D)	(None, 20, 20, 256)	590,080
activation_5 (Activation)	(None, 20, 20, 256)	0
conv2d_6 (Conv2D)	(None, 20, 20, 256)	590,080
activation_6 (Activation)	(None, 20, 20, 256)	0
max_pooling2d_2 (MaxPooling2D)	(None, 10, 10, 256)	0

conv2d_7 (Conv2D)	(None, 10, 10, 512)	1,180,160
activation_7 (Activation)	(None, 10, 10, 512)	0
conv2d_8 (Conv2D)	(None, 10, 10, 512)	2,359,808
activation_8 (Activation)	(None, 10, 10, 512)	0
conv2d_9 (Conv2D)	(None, 10, 10, 512)	2,359,808
activation_9 (Activation)	(None, 10, 10, 512)	0
max_pooling2d_3 (MaxPooling2D)	(None, 5, 5, 512)	0
conv2d_10 (Conv2D)	(None, 5, 5, 512)	2,359,808
activation_10 (Activation)	(None, 5, 5, 512)	0
conv2d_11 (Conv2D)	(None, 5, 5, 512)	2,359,808
activation_11 (Activation)	(None, 5, 5, 512)	0
conv2d_12 (Conv2D)	(None, 5, 5, 512)	2,359,808
activation_12 (Activation)	(None, 5, 5, 512)	0
max_pooling2d_4 (MaxPooling2D)	(None, 2, 2, 512)	0
flatten (Flatten)	(None, 2048)	0
dense (Dense)	(None, 4096)	8,392,704
activation_13 (Activation)	(None, 4096)	0
dense_1 (Dense)	(None, 4096)	16,781,312
activation_14 (Activation)	(None, 4096)	0
dense_2 (Dense)	(None, 2)	8,194
activation_15 (Activation)	(None, 2)	0

Table 1. VGG16 Structure

4. Regularization

To avoid wasting time on ineffective learning, an EarlyStopping callback was added, with patience set to 10 — training is forcibly stopped if the validation loss (val_loss) does not decrease for 10 consecutive epochs.

```
early_stopping_callback = EarlyStopping(monitor='val_loss',
                                        patience=10,
                                        verbose=1,
                                        restore_best_weights=True)
```

Figure 3. EarlyStopping callback parameters

In addition, to prevent overfitting, L1 regularization and L2 regularization were added to the first convolutional layer and the fully connected layers of the VGG16 model.

L1 regularization formula: $Loss = Error(y, \hat{y}) + \lambda \Sigma |W_i|$

L2 regularization formula: $Loss = Error(y, \hat{y}) + \lambda \Sigma W_i^2$

(where y is the actual value, ŷ is the predicted value, λ is the regularization parameter, and W is the model weight [3].)

5. Dynamic Learning Rate

To find the optimal model parameters, this study combined Adaptive Moment Estimation (Adam) with learning-rate callbacks (ReduceLROnPlateau and LearningRateScheduler) to search for the best learning rate.

Adam is an adaptive learning-rate optimization algorithm that combines RMSprop with momentum-based gradient descent. ReduceLROnPlateau is a built-in Keras method; in this study, patience=5, meaning that if val_loss does not decrease for more than 5 steps, the new learning rate is computed as lr × factor, with a minimum learning rate of min_lr.

```
reduce_lr = ReduceLROnPlateau(monitor='val_loss',
                              factor=0.2,
                              patience=5,
                              min_lr=0.00001)
```

Figure 4. ReduceLROnPlateau parameter settings [4]

LearningRateScheduler is also a built-in Keras method that allows a custom dynamic learning-rate function to be defined. In this study, whenever the step count exceeds 5, lr = lr × 0.1, gradually lowering the learning rate to achieve better convergence.

```
def scheduler(epoch, lr):
    if epoch % 5 == 0 and epoch != 0:
        return lr * 0.1
    else:
        return lr
learning_rate_scheduler = LearningRateScheduler(scheduler)
```

Figure 5. LearningRateScheduler parameter settings [5]

6. Binary Cross-Entropy

Binary cross-entropy loss is typically used for binary classification problems — in this case, classifying an input into one of two categories (synthetic image or real image). It measures the difference between the predicted probability and the true label. The binary cross-entropy loss formula is as follows:

$L = \Sigma_i \; y_i \cdot \log(O_i) + (1 - y_i) \cdot \log(1 - O_i)$

(where y represents the true label — 0 or 1 — and O_i represents the predicted probability of the positive class.)

IV. Results and Discussion

1. VGG16 Model Training Results on the CIFake Dataset

(1) Training with the Plain VGG16 Model

1. Purpose: To serve as a control group prior to adding regularization and callback parameters.

2. Parameter settings: Input layer size changed to 80×80 (originally 225×225) to match this experiment's requirements; epochs = 80; no callback functions used.

3. Results:

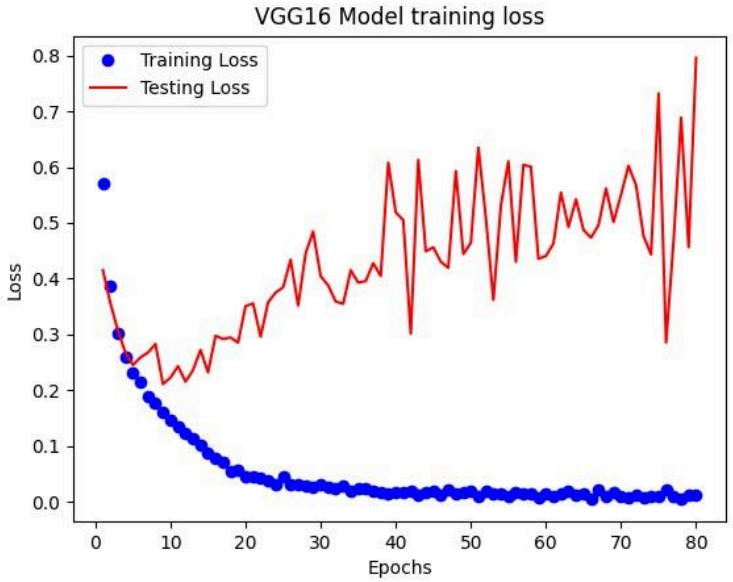


Figure 6. VGG16 model training results on the CIFake dataset

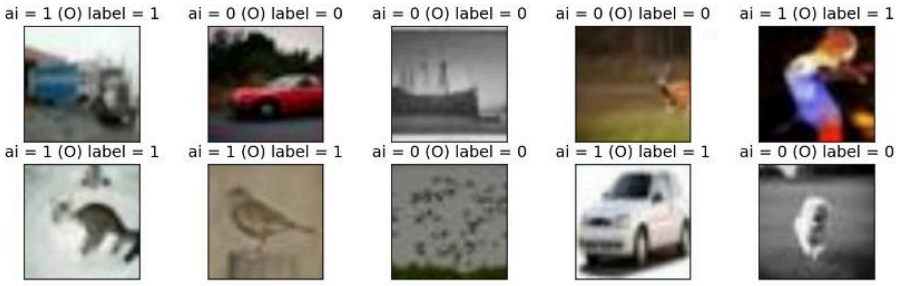


Figure 7. Sample of the first ten prediction outputs

4. Discussion: Without the help of regularization or a dynamic learning rate, the model exhibited overfitting, and the testing loss already showed an upward trend by around epoch 5.

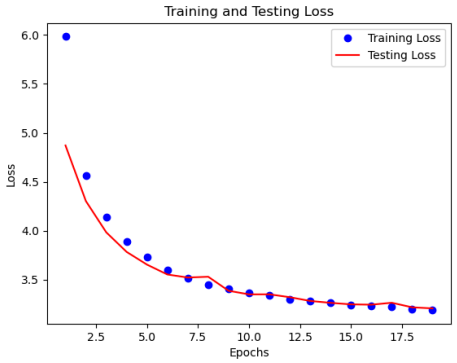
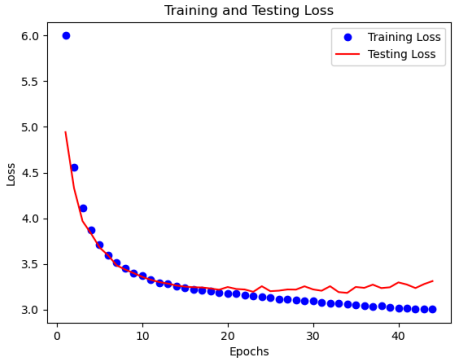
2. Regularization and Dynamic Learning Rate

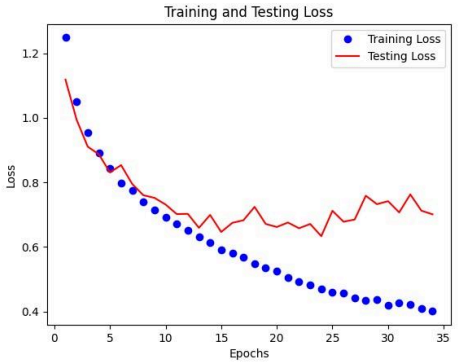
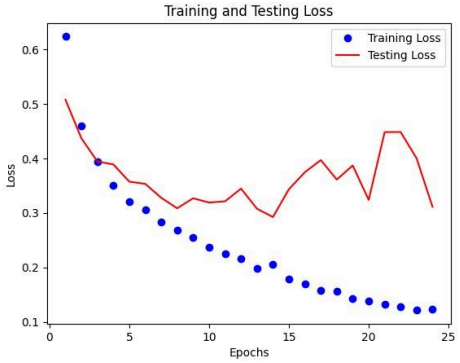
(1) Varying Regularization

Regularization is used in machine learning and deep learning to address the problem of overfitting. L1 and L2 regularization are among the most common regularization techniques. They add a penalty term to the loss function to constrain the model's weights, thereby reducing overfitting. Because the fluctuations (and loss

values) are large in the early stages of training, all charts below begin plotting from epoch 2. The table below shows results using only L1/L2 regularization.

Parameter settings: Input size (80, 80, 3), lr = 0.0001, batch size = 32:

No.	Early-stoppin gepoch	L1 / L2	Testing accuracy /Testing loss	Loss-epoch chart
1	Epoch = 20	L1 = 0.01 L2 = 0.01	Accuracy = 0.9207 Loss = 3.206	
2	Epoch = 45	L1 = 0.01 L2 = 0.0001	Accuracy = 0.9210 Loss = 3.181	
3	Epoch = 32	L1 = 0.001 L2 = 0.001	Accuracy = 0.9265 Loss = 0.6196	

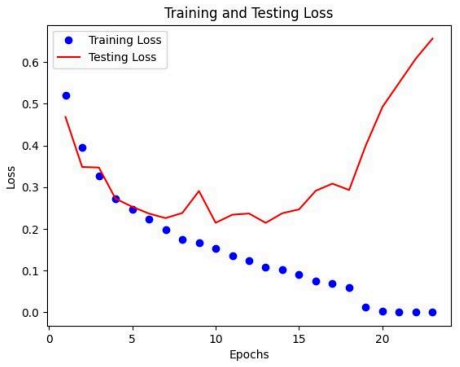
4	Epoch = 35	L1 = 0.001 L2 = 0.0001	Accuracy = 0.9300 Loss = 0.6066	
5	Epoch = 25	L1 = 0.0001 L2 = 0.0001	Accuracy = 0.9327 Loss = 0.2602	

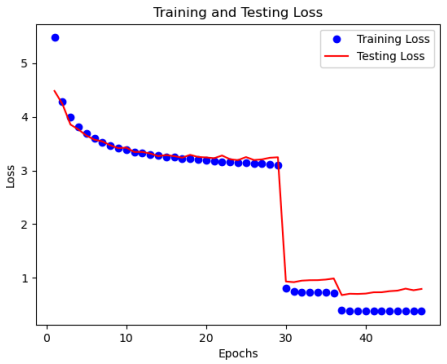
Discussion: As L1 and L2 values decrease, testing accuracy increases. Although there is a slight overfitting effect, the gap between testing loss and training loss is not large, and is therefore considered within an acceptable range.

(2) Varying the Dynamic Learning Rate

A dynamic learning rate automatically adjusts during different training stages to control the model's convergence speed. This is because, during training, a learning rate that is too large may cause the model to overshoot the optimal solution, while a learning rate that is too small results in very slow convergence. Therefore, dynamically adjusting the learning rate helps the model converge better. The table below shows results using only the callback functions.

Parameter settings: Input size (80, 80, 3), lr = 0.0001, batch size = 32:

No.	**Early-stoppingepoch**	**Callback**	**Testing accuracy /Testing loss**	**Loss-epoch chart**
6	Epoch = 23	scheduler	Accuracy = 0.9185 Loss = 0.2268	

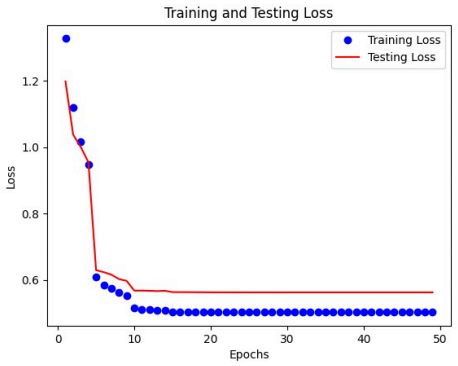
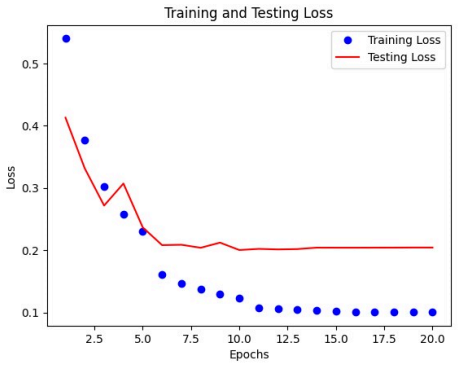
7	Epoch = 20	Reduce_lr	Accuracy = 0.9292 Loss = 0.1868	

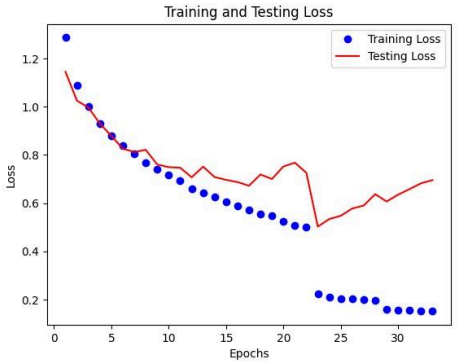
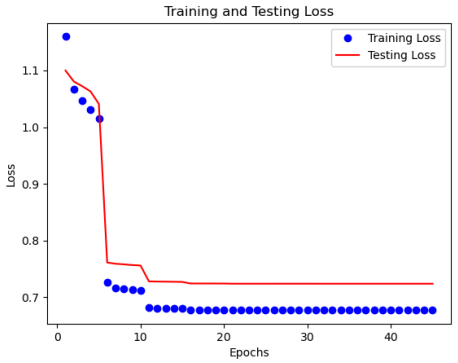
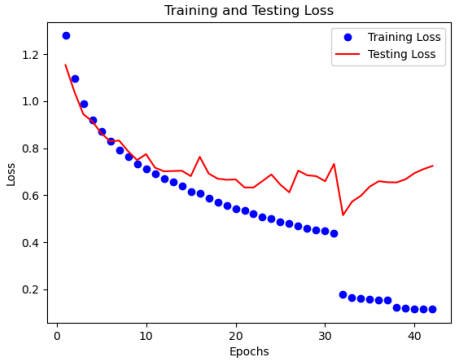
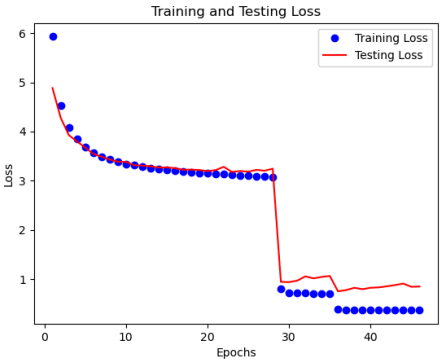
Discussion: The two charts above show that adding a callback function slightly reduced the original VGG16 loss value, with reduce_lr producing better results. From the chart for No. 6, the testing loss already shows an upward trend by around epoch 7, because with the scheduler the learning rate only decreases once every 5 steps, which is not as responsive as ReduceLROnPlateau.

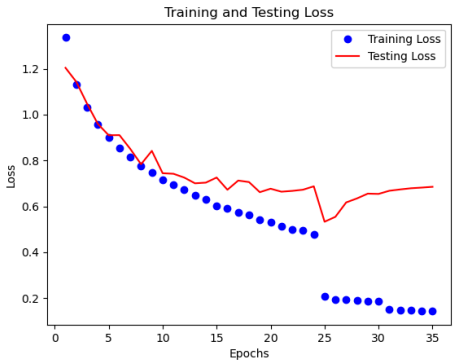
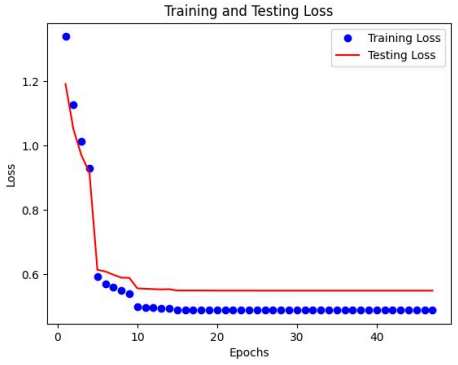
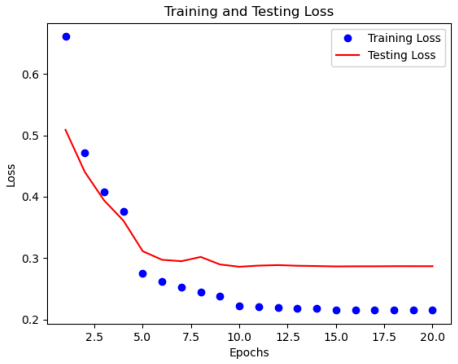
(3) Varying Both Regularization and Dynamic Learning Rate

The two sets of results above show that both regularization and dynamic learning rate can reduce the degree of model overfitting. Therefore, by combining regularization (to prevent overfitting) with a dynamic learning rate (to improve training speed and performance), the most suitable model can be found. The table below shows results using both regularization and dynamic learning-rate adjustment together.

Parameter settings: Input size (80, 80, 3), lr = 0.0001, batch size = 32:

No.	Early-stoppingepoch	L1 / L2 / Callback	Testing accuracy /Testing loss	Loss-epoch chart
8	Epoch = 48	L1 = 0.01 L2 = 0.01 Reduce_lr	Accuracy = 0.9277 Loss = 0.6946	
9	Epoch = 50	L1 = 0.01 L2 = 0.01 scheduler	Accuracy = 0.9287 Loss = 0.5490	

10	Epoch = 34	L1 = 0.01 L2 = 0.0001 Reduce_lr	Accuracy = 0.9312 Loss = 0.4769	
11	Epoch = 50	L1 = 0.01 L2 = 0.0001 scheduler	Accuracy = 0.9112 Loss = 0.7089	
12	Epoch = 43	L1 = 0.001 L2 = 0.001 Reduce_lr	Accuracy = 0.9369 Loss = 0.4741	
13	Epoch = 47	L1 = 0.001 L2 = 0.001 scheduler	Accuracy = 0.9367 Loss = 0.7016	

14	Epoch = 36	L1 = 0.001 L2 = 0.0001 Reduce_lr	Accuracy = 0.9359 Loss = 0.4923	
15	Epoch = 48	L1 = 0.001 L2 = 0.0001 scheduler	Accuracy = 0.9294 Loss = 0.5357	
16	Epoch = 22	L1 = 0.0001 L2 = 0.0001 Reduce_lr	Accuracy = 0.9272 Loss = 0.2821	
17	Epoch = 21	L1 = 0.0001 L2 = 0.0001 scheduler	Accuracy = 0.9309 Loss = 0.2699	

Discussion: Although No. 12 achieved the highest testing accuracy at 0.9369, its loss value was noticeably higher than that of No. 13, whose testing accuracy was also high at 0.9367. Therefore, the final chosen parameter settings were: L1 = 0.001, L2 = 0.001, Callback = scheduler, Epochs = 47.

3. Comparing the Custom Model with Existing AI Image-Detection Websites

(1) Image Selection

Publicly available images from MidJourney and Stable Diffusion were used to find images with minimal visualized noise — ones that AI image-detection platforms would fail to correctly identify — for testing. The figure below shows the test image, followed by the results from each platform.



Figure 8. An AI-generated image that causes detection platforms to misclassify it

1. Optic AI or Not

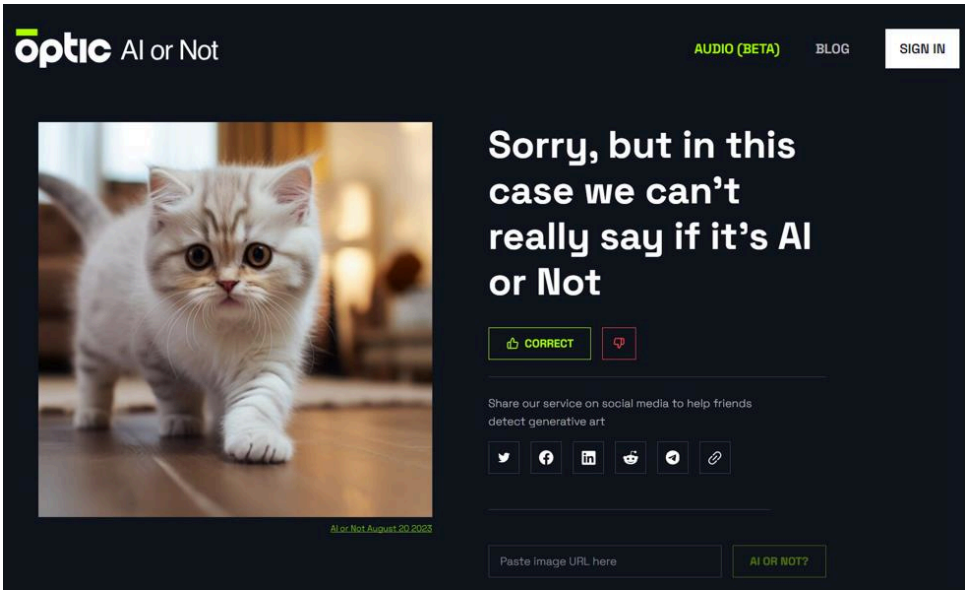


Figure 9. Optic AI or Not fails to correctly identify the image

2. Illuminarty

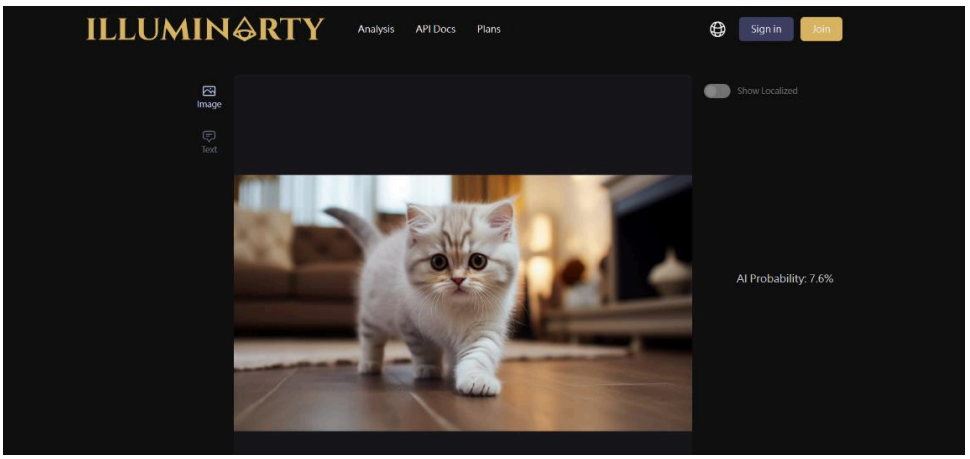


Figure 10. Illuminarty judges the image as only 7.6% likely to be AI-generated

3. Maybe's AI Art Detector

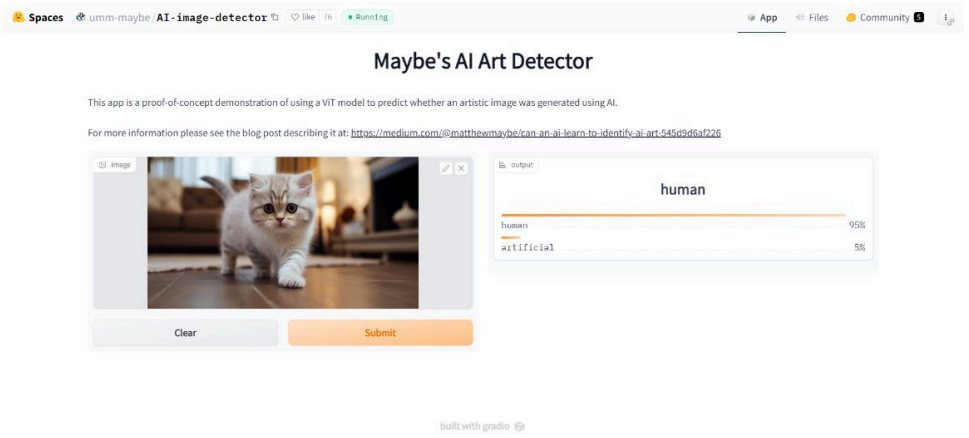


Figure 11. Maybe's AI Art Detector judges the image as only 5% likely to be AI-generated

4. The Custom VGG16 Model

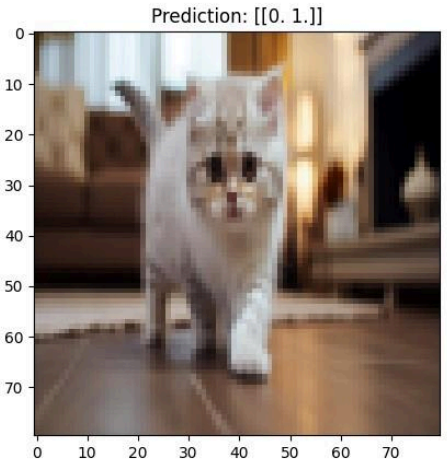


Figure 12. The custom VGG16 model successfully identifies the image as AI-generated

V. Conclusion

This study found that adding regularization and a dynamic learning rate to the VGG16 model can reduce model overfitting and effectively increase accuracy. With the parameter settings L1 = 0.001, L2 = 0.001, Callback = scheduler, and Epochs = 47, the model showed only a small degree of overfitting, achieving a testing accuracy as high as 0.9367. This result demonstrates that the model has the ability to detect non-visualized noise.

By comparing the custom model against online AI image-detection platforms, this study successfully identified AI-generated images that the other platforms were unable to correctly identify.

VI. Future Outlook

(1) Trying Other Models

Within the VGG-Net family, there are many other models and derivative architectures, such as VGG19, as well as newer models. We hope to try these in future research.

(2) Dataset Optimization

The dataset used in this project consisted of images that were only 32×32 pixels, which limited the achievable accuracy due to insufficient detail. We hope to conduct future research using a higher-resolution dataset.

VII. References

[1] Simonyan, K., & Zisserman, A. (2014). Very Deep Convolutional Networks for Large-Scale Image Recognition. arXiv preprint arXiv:1409.1556.

[2] Birdy654. (n.d.). CIFAKE: Real and AI-Generated Synthetic Images. Kaggle. https://www.kaggle.com/datasets/birdy654/cifake-real-and-ai-generated-synthetic-images/data

[3] Stack Overflow. (2019). Compute the Loss of L1 and L2 regularization. Retrieved from https://stackoverflow.com/questions/58905671/compute-the-loss-of-l1-and-l2-regularization

[4] Keras. (n.d.). ReduceLROnPlateau. Keras API documentation. Retrieved from https://keras.io/api/callbacks/reduce_lr_on_plateau/

[5] Keras. (n.d.). LearningRateScheduler. Keras API documentation. Retrieved from https://keras.io/api/callbacks/learning_rate_scheduler/